

SWE-642: ASSIGNMENT 3

Dhruvi Pruthvisinh Rathod - G01465151

Namita Chougule – G01473740

Swarangi Kunbi - G01487325

Aditya Shah - G01449892

Frontend

This section explains the structure, functionality, and key features of the Angular application.

1. Overview

The frontend application is developed using Angular and includes:

- **Components:**
 - `create-student.component`: Used to fill and submit the student survey form.
 - `student-list.component`: Displays the list of all student surveys.
 - `update-student.component`: Allows users to update an existing student survey.
- **Routing:** Configured to navigate between components using `routerLink`.

2. Application Flow

1. The user accesses the **Student Survey List** or **Fill Student Survey** via the navigation bar in `app.component.html`(`app.component`).
2. Each route points to a specific component:
 - `/survey` → `create-student.component` for adding surveys.
 - `/surveys` → `student-list.component` for viewing and managing surveys.

3. Core Functionalities

- **Student Survey Form (`create-student.component.html`):**
 - Collects user input for fields such as name, address, city, state, email, and survey details(`create-student.component`).
 - Features include:
 - Two-way data binding with `[(ngModel)]`.
 - Form validation (`required` for mandatory fields).
 - Submission handled by `onSubmit()` in `create-student.component.ts`.
- **Student List (`student-list.component.html`):**

- Displays a table of all submitted surveys, fetched from the backend via an API (student-list.component).
- Includes action buttons for updating and deleting records.
- **Update Survey Form (update-employee.component.html):**
 - Allows editing survey details using a pre-populated form fetched from the backend(update-employee.component).

4. Key Features

- Uses Bootstrap for responsive UI design.
- Implements Angular's `RouterModule` for navigation.
- Data operations (add, update, delete) are performed using HTTP requests handled by Angular services.

5. Installation and Execution

1.Install dependencies:

`npm install`

2.Start the application:

`ng serve`

3.Open the app at <http://localhost:4200>.

Backend

This section explains the structure, endpoints, and key features of the Spring Boot backend.

1. Overview

The backend application is developed using Spring Boot and includes:

- **RESTful APIs** for CRUD operations on student surveys.
 - **JPA/Hibernate** for database interaction.
-

2. Application Flow

1. Swe642Homework3BackendApplication.java

- **Purpose:**
 - The main entry point of the Spring Boot application.
 - It contains the `main()` method that bootstraps and launches the application.
- **Key Responsibilities:**
 - Starts the Spring Boot application by invoking `SpringApplication.run()`.

2. application.properties

- **Purpose:**
 - Configures the database connection and Hibernate behavior.
- **Key Configurations:**
 - `spring.datasource.url`: Defines the URL for the MySQL database.
 - `spring.datasource.username` and `spring.datasource.password`: Credentials for connecting to the database.
 - `spring.jpa.hibernate.ddl-auto`: Automatically updates the database schema based on entity definitions.

3. Student.java

- **Purpose:**
 - Represents the `Student` entity mapped to the `surveys` table in the database.
- **Annotations:**
 - `@Entity`: Marks the class as a JPA entity.
 - `@Table`: Specifies the database table (`surveys`).
 - `@Id` and `@GeneratedValue`: Indicate the primary key and auto-increment strategy.
- **Fields:**

- `firstName`, `lastName`, `streetAddress`, `email`, `likedOptions`, etc.
- **Relationships:**
 - Each field corresponds to a column in the `surveys` table.

4. StudentController.java

- **Purpose:**
 - Exposes RESTful endpoints to interact with the `Student` entity.
- **Key Endpoints:**
 - `@GetMapping("/api/v1/surveys")`: Retrieves all surveys.
 - `@PostMapping("/api/v1/survey")`: Creates a new survey.
 - `@PutMapping("/api/v1/surveys/{id}")`: Updates an existing survey by `id`.
 - `@DeleteMapping("/api/v1/surveys/{id}")`: Deletes a survey by `id`.
- **Annotations:**
 - `@RestController`: Marks the class as a RESTful controller.
 - `@RequestMapping`: Defines the base path for all endpoints.

5. StudentRepository.java

- **Purpose:**
 - Interface for interacting with the database using JPA.
- **Key Features:**
 - Extends `JpaRepository<Student, Long>`, providing built-in methods like `findAll()`, `save()`, `deleteById()`, etc.
 - Abstracts database operations, so no SQL queries are needed in the controller.

6. ResourceNotFoundException.java

- **Purpose:**
 - Custom exception for handling cases where a requested resource (e.g., a student by `id`) is not found in the database.
 - **Annotations:**
 - `@ResponseStatus(HttpStatus.NOT_FOUND)`: Automatically returns a 404 response when this exception is thrown.
 - **Usage:**
 - Used in the `StudentController` to handle invalid `id` lookups.
-

3. Core Functionalities

- **CRUD Operations:**
 - **GET `/api/v1/surveys`**: Fetch all surveys.

- **POST** `/api/v1/survey`: Add a new survey.
 - **PUT** `/api/v1/surveys/{id}`: Update an existing survey.
 - **DELETE** `/api/v1/surveys/{id}`: Delete a survey.
 - **Entity Definition (`Student.java`):**
 - Fields include `firstName`, `lastName`, `streetAddress`, `email`, `likedOptions`, etc.
 - Annotations like `@Entity` and `@Table` map the class to the database table.
 - **Exception Handling:**
 - Uses `ResourceNotFoundException` to handle invalid IDs.
 - **CORS Configuration:**
 - Configured to allow requests from the Angular frontend (`http://localhost:4200`).
-

4. Installation and Execution

1. Install dependencies:

```
mvn install
```

2. Run the application:

```
mvn spring-boot:run
```

3. Verify APIs via Postman or Angular frontend.

User Interface :

1. Survey Form

STUDENT SURVEY LIST

FILL STUDENT SURVEY

Student Survey Form

CS Department Survey

First Name: *

First Name is required.

Last Name: *

Street Address Line 1: *

City: *

eg: Fairfax

State: *

eg: VA

Zipcode: *

eg: 22030

2. Collected Surveys

STUDENT SURVEY LIST

FILL STUDENT SURVEY

Student's List

First Name	Last Name	Email	Street Address	City	State	Zipcode	telephone	Date	likedOptions	likelihoodToRe
sdfvg	desfdgf	adf@qw.sd	ewretry	Fairfax	123456	sdf	12345678909	2024-11-30	Students: Yes	likely
wesrdf	eregtrh	qwert@qwert.zxc	q34rtyu	qwerth	123456	VA	12345678909	2024-11-30	Students: Yes	very_lik
asdfgh	dfgh	asdf@as.xc	qwertyuiopijhgv	Fairfax	22030	VA	123454321	2024-12-01	Atmosphere: Yes	likely
sd	sdf	wer@qwe.cvb	erdgf	rgfhg	2345	retg	32456768654	2024-12-01	Dorms: Yes	very_lik

3. Delete And Update Button

STUDENT SURVEY LIST

FILL STUDENT SURVEY

Student's List

ess	City	State	Zipcode	telephone	Date	likedOptions	likelihoodToRecommend	sourceOfInterest	comments	Actions
	Fairfax	123456	sdf	12345678909	2024-11-30	Students: Yes	likely	internet		UpdateDelete
	qwerth	123456	VA	12345678909	2024-11-30	Students: Yes	very_likely	friends		UpdateDelete
ijhgv	Fairfax	22030	VA	123454321	2024-12-01	Atmosphere: Yes	likely	internet		UpdateDelete
	rgfhg	2345	retg	32456768654	2024-12-01	Dorms: Yes	very_likely	friends		UpdateDelete

4. Database After Deleting A Survey

localhost

MyS...SQL...You...[no...swe...stir...CGI...Wha...[no...swe...Geo...Swe842...642...Sha...

STUDENT SURVEY LIST

FILL STUDENT SURVEY

Student's List

First Name	Last Name	Email	Street Adress	City	State	Zipcode	telephone	Date	likedOptions	likelihoodToRe
wesrdf	eregtrh	qwert@qwert.zxc	q34rtyu	qwerth	123456	VA	12345678909	2024-11-30	Students: Yes	very_like
asdfgh	dfgh	asdf@as.xc	qwertyuiopijhgv	Fairfax	22030	VA	123454321	2024-12-01	Atmosphere: Yes	likely
sd	sdf	wer@qwe.cvb	erdgf	rgfhg	2345	retg	32456768654	2024-12-01	Dorms: Yes	very_like